

Day 1 과제

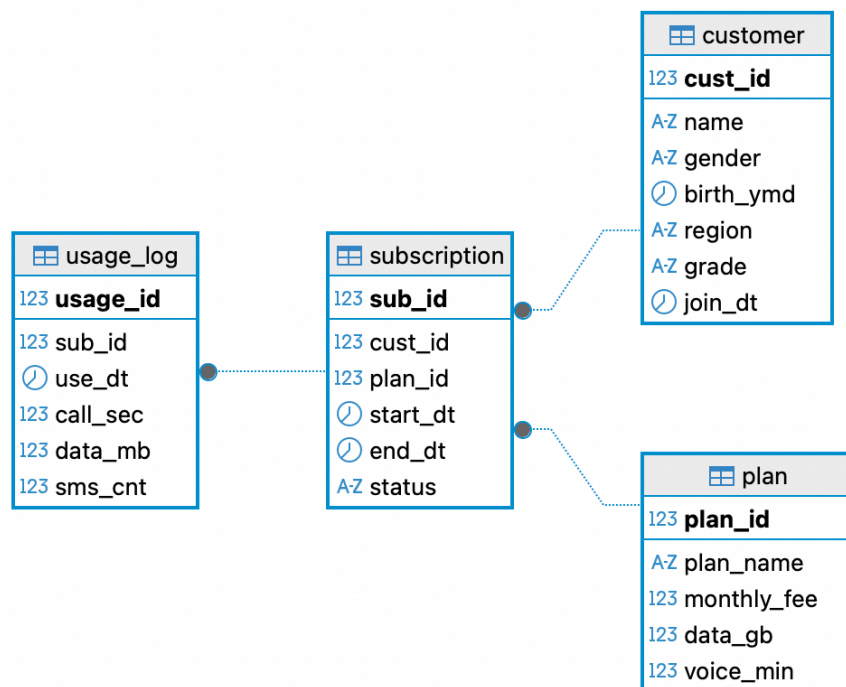
1. 표지

- 이름: 김준수
- 과정명: AI 서비스를 위한 SW 기초(Full-stack Engineering)
- 제출일: 2026-07-20
- 실습 주제: Day1 — 환경 구축·데이터 모델링·SQL 기초 (telecom 가입자 관리 DB)

2. 요구사항 요약 — 설계 방향·가정

- **시나리오:** 통신사 가입자 관리 시스템의 DB(telecom)를 설계·구축한다. 고객(customer)·요금제(plan)·가입(subscription)·사용로그(usage_log) 4개 테이블로 구성된다.
- **설계 방향**
 - customer는 고객 1명당 1행(마스터), plan은 요금제 1종당 1행(마스터)로 둔다.
 - subscription은 고객과 요금제 간 **다대다 이력**을 표현하는 연결 테이블로, 한 고객이 여러 번 가입/해지할 수 있으므로 cust_id를 FK로 두되 PK는 별도 서로게이트 키(sub_id)로 둔다.
 - usage_log는 가입 건별 일별 사용량을 기록하는 트랜잭션성 테이블로, 볼륨이 가장 크므로 인덱스·튜닝은 Day3에서 다룬다.
- **가정**
 - end_dt IS NULL이면 현재 사용 중인 가입으로 간주한다.
 - region, gender는 필수값이 아닐 수 있어 NULL을 허용한다.
 - grade는 기본값 SILVER이며 BRONZE/SILVER/GOLD/VIP 4단계로 관리한다.

3. ERD 이미지



범례: 사각형 = 테이블, 굵은 컬럼 = PK(기본키) · 선 끝 갈매기(●) = FK를 가진 자식(N) 쪽, 반대편(선만) = 부모(1) 쪽 → 1:N 관계

관계 요약

관계	부모(1)	자식(N)	FK 컬럼
고객-가입	customer	subscription	subscription.cust_id → customer.cust_id
요금제-가입	plan	subscription	subscription.plan_id → plan.plan_id
가입-사용로그	subscription	usage_log	usage_log.sub_id → subscription.sub_id

4. 정규화 워크시트 — 과제 1

대상 비정규화 표 (주문 내역, PK = order_id + prod_id)

order_id	order_dt	cust_id	cust_name	prod_id	prod_name	unit_price	qty
O1001	2024-06-01	C01	김하늘	P10	노트북	1200000	1
O1001	2024-06-01	C01	김하늘	P22	마우스	25000	2
O1002	2024-06-03	C02	이준호	P10	노트북	1200000	1

```
CREATE TABLE order_raw (
  order_id  VARCHAR(10) NOT NULL,
  order_dt  DATE        NOT NULL,
  cust_id   VARCHAR(10) NOT NULL,
  cust_name VARCHAR(50) NOT NULL,
  prod_id   VARCHAR(10) NOT NULL,
  prod_name VARCHAR(50) NOT NULL,
  unit_price INTEGER     NOT NULL,
  qty       INTEGER     NOT NULL,
  PRIMARY KEY (order_id, prod_id)
);

INSERT INTO order_raw
  (order_id, order_dt, cust_id, cust_name, prod_id, prod_name, unit_price, qty)
VALUES
  ('O1001', DATE '2024-06-01', 'C01', '김하늘', 'P10', '노트북', 1200000, 1),
  ('O1001', DATE '2024-06-01', 'C01', '김하늘', 'P22', '마우스', 25000, 2),
  ('O1002', DATE '2024-06-03', 'C02', '이준호', 'P10', '노트북', 1200000, 1);

-- 캡처①: 1NF 원본 표 (모든 칸이 원자값, 반복 그룹 없음)
SELECT * FROM order_raw ORDER BY order_id, prod_id;
```

```

CREATE TABLE order_raw (
  order_id VARCHAR(10) NOT NULL,
  order_dt DATE NOT NULL,
  cust_id VARCHAR(10) NOT NULL,
  cust_name VARCHAR(50) NOT NULL,
  prod_id VARCHAR(10) NOT NULL,
  prod_name VARCHAR(50) NOT NULL,
  unit_price INTEGER NOT NULL,
  qty INTEGER NOT NULL,
  PRIMARY KEY (order_id, prod_id)
);

INSERT INTO order_raw
  (order_id, order_dt, cust_id, cust_name, prod_id, prod_name, unit_price, qty)
VALUES
  ( 'order_id' '01001', 'order_dt' DATE '2024-06-01', 'cust_id' 'C01', 'cust_name' '김하늘', 'prod_id' 'P10', 'prod_name' '노트북', 'unit_price' 1200000, 'qty' 1),
  ( 'order_id' '01001', 'order_dt' DATE '2024-06-01', 'cust_id' 'C01', 'cust_name' '김하늘', 'prod_id' 'P22', 'prod_name' '마우스', 'unit_price' 25000, 'qty' 2),
  ( 'order_id' '01002', 'order_dt' DATE '2024-06-03', 'cust_id' 'C02', 'cust_name' '이준호', 'prod_id' 'P10', 'prod_name' '노트북', 'unit_price' 1200000, 'qty' 1);

-- 검색: 1NF 원본 표 (모든 칸이 원자값, 반복 그룹 없음)
SELECT * FROM order_raw ORDER BY order_id, prod_id;

```

① 함수 종속 식별

종속 관계	의미
order_id → order_dt, cust_id	주문 단위 속성은 주문번호에만 종속
cust_id → cust_name	고객명은 고객번호에 종속(이행 종속)
prod_id → prod_name, unit_price	상품 단위 속성은 상품번호에만 종속
(order_id, prod_id) → qty	수량은 "어느 주문의 어느 상품인지"(복합키) 전체에 종속

② 1NF → 2NF → 3NF 분해

1NF 확인: 모든 칸이 원자값이고 반복 그룹이 없으므로 주어진 표는 이미 1NF를 만족한다.

1NF → 2NF (부분 종속 제거)

PK가 (order_id, prod_id) 복합키인데, order_dt·cust_id·cust_name은 order_id에만, prod_name·unit_price는 prod_id에만 종속 되는 **부분 종속**이 존재 → 분리한다.

- ORDERS(order_id PK, order_dt, cust_id, cust_name)
- PRODUCTS(prod_id PK, prod_name, unit_price)
- ORDER_ITEMS(order_id, prod_id, qty) — PK(order_id, prod_id)

2NF 단계 실제 데이터 — 원본 3행(O1001×2, O1002×1)을 위 3개 테이블로 나눠놓으면 아래와 같다. (이 단계에서는 ORDERS에 아직 cust_name이 남아있어 이행 종속이 해소되지 않은 상태다.)

```

CREATE TABLE orders_2nf (
  order_id VARCHAR(10) PRIMARY KEY,
  order_dt DATE NOT NULL,
  cust_id VARCHAR(10) NOT NULL,
  cust_name VARCHAR(50) NOT NULL
);

CREATE TABLE products_2nf (
  prod_id VARCHAR(10) PRIMARY KEY,
  prod_name VARCHAR(50) NOT NULL,
  unit_price INTEGER NOT NULL
);

CREATE TABLE order_items_2nf (
  order_id VARCHAR(10) NOT NULL REFERENCES orders_2nf(order_id),
  prod_id VARCHAR(10) NOT NULL REFERENCES products_2nf(prod_id),
  qty INTEGER NOT NULL,
  PRIMARY KEY (order_id, prod_id)
);

```

```
);

INSERT INTO orders_2nf (order_id, order_dt, cust_id, cust_name) VALUES
  ('01001', DATE '2024-06-01', 'C01', '김하늘'),
  ('01002', DATE '2024-06-03', 'C02', '이준호');

INSERT INTO products_2nf (prod_id, prod_name, unit_price) VALUES
  ('P10', '노트북', 1200000),
  ('P22', '마우스', 25000);

INSERT INTO order_items_2nf (order_id, prod_id, qty) VALUES
  ('01001', 'P10', 1),
  ('01001', 'P22', 2),
  ('01002', 'P10', 1);

-- 캡처②: 2NF 단계 3개 테이블
SELECT * FROM orders_2nf ORDER BY order_id;
SELECT * FROM products_2nf ORDER BY prod_id;
SELECT * FROM order_items_2nf ORDER BY order_id, prod_id;
```

ORDERS (2NF 단계) — PK: order_id

order_id (PK)	order_dt	cust_id	cust_name
O1001	2024-06-01	C01	김하늘
O1002	2024-06-03	C02	이준호

-- 캡처②: 2NF 단계 3개 테이블

SELECT * FROM orders_2nf ORDER BY order_id; 340 ms

Output 캡처②: 2NF 단계 3개 테이블 ×

	order_id ▼	order_dt ▼	cust_id ▼	cust_name ▼
1	01001	2024-06-01	C01	김하늘
2	01002	2024-06-03	C02	이준호

PRODUCTS (2NF 단계) — PK: prod_id

prod_id (PK)	prod_name	unit_price
P10	노트북	1200000
P22	마우스	25000

-- 캡처②: 2NF 단계 3개 테이블

SELECT * FROM orders_2nf ORDER BY order_id; 340 ms

Output 캡처②: 2NF 단계 3개 테이블 ×

	order_id ▼	order_dt ▼	cust_id ▼	cust_name ▼
1	01001	2024-06-01	C01	김하늘
2	01002	2024-06-03	C02	이준호

ORDER_ITEMS (2NF 단계) — PK: (order_id, prod_id)

order_id (PK)	prod_id (PK)	qty
O1001	P10	1
O1001	P22	2

order_id (PK)	prod_id (PK)	qty
O1002	P10	1

The screenshot shows a SQL IDE with a query window and an output window. The query is `SELECT * FROM order_items_2nf ORDER BY order_id, prod_id;` and it took 334 ms to execute. The output window shows a table with 3 rows and 4 columns: `order_id`, `prod_id`, `qty`, and a tooltip for `prod_id: varchar(10)`.

	order_id	prod_id	qty
1	01001	P10	1
2	01001	P22	2
3	01002	P10	1

2NF → 3NF (이행 종속 제거)

ORDERS에서 order_id → cust_id → cust_name으로 **이행 종속**이 존재(키가 아닌 속성이 다른 키가 아닌 속성에 종속) → cust_name을 분리한다.

- CUSTOMERS(cust_id PK, cust_name)
- ORDERS(order_id PK, order_dt, cust_id FK → CUSTOMERS)
- PRODUCTS(prod_id PK, prod_name, unit_price) — 상품 속성 간 이행 종속 없음, 그대로 유지
- ORDER_ITEMS(order_id FK → ORDERS, prod_id FK → PRODUCTS, qty) — PK(order_id, prod_id)

3NF 단계 실제 데이터 — 2NF 단계의 ORDERS에서 cust_name이 CUSTOMERS로 빠져나간 결과는 아래와 같다. (PRODUCTS-ORDER_ITEMS는 2NF 단계와 동일하므로 변화가 있는 CUSTOMERS-ORDERS만 표시)

```
CREATE TABLE customers (
  cust_id VARCHAR(10) PRIMARY KEY,
  cust_name VARCHAR(50) NOT NULL
);

CREATE TABLE products (
  prod_id VARCHAR(10) PRIMARY KEY,
  prod_name VARCHAR(50) NOT NULL,
  unit_price INTEGER NOT NULL
);

CREATE TABLE orders (
  order_id VARCHAR(10) PRIMARY KEY,
  order_dt DATE NOT NULL,
  cust_id VARCHAR(10) NOT NULL REFERENCES customers(cust_id)
);

CREATE TABLE order_items (
  order_id VARCHAR(10) NOT NULL REFERENCES orders(order_id),
  prod_id VARCHAR(10) NOT NULL REFERENCES products(prod_id),
  qty INTEGER NOT NULL,
  PRIMARY KEY (order_id, prod_id)
);

INSERT INTO customers (cust_id, cust_name) VALUES
  ('C01', '김하늘'),
  ('C02', '이준호');
```

```

INSERT INTO products (prod_id, prod_name, unit_price) VALUES
  ('P10', '노트북', 1200000),
  ('P22', '마우스', 25000);

INSERT INTO orders (order_id, order_dt, cust_id) VALUES
  ('O1001', DATE '2024-06-01', 'C01'),
  ('O1002', DATE '2024-06-03', 'C02');

INSERT INTO order_items (order_id, prod_id, qty) VALUES
  ('O1001', 'P10', 1),
  ('O1001', 'P22', 2),
  ('O1002', 'P10', 1);

-- 캡처③: 3NF 최종 4개 테이블
SELECT * FROM customers ORDER BY cust_id;
SELECT * FROM products ORDER BY prod_id;
SELECT * FROM orders ORDER BY order_id;
SELECT * FROM order_items ORDER BY order_id, prod_id;

```

CUSTOMERS (신규 분리) — PK: cust_id

cust_id (PK)	cust_name
C01	김하늘
C02	이준호

```

SELECT * FROM customers ORDER BY cust_id; 330 ms

```

cust_id	cust_name
1 C01	김하늘
2 C02	이준호

ORDERS (3NF 단계, cust_name 제거됨) — PK: order_id · FK: cust_id → CUSTOMERS.cust_id

order_id (PK)	order_dt	cust_id (FK)
O1001	2024-06-01	C01
O1002	2024-06-03	C02

```

SELECT * FROM orders ORDER BY order_id; 372 ms

```

order_id	order_dt	cust_id
1 O1001	2024-06-01	C01
2 O1002	2024-06-03	C02

③ 최종 테이블 · PK·FK 정리 (데이터 포함)

원본 3행(O1001×2, O1002×1)을 4개 테이블로 분해한 최종 결과. 테이블별 PK·FK는 표 제목 아래에 명시.

CUSTOMERS — PK: cust_id

cust_id (PK)	cust_name
C01	김하늘
C02	이준호

SELECT * FROM customers ORDER BY cust_id; 330 ms

Output: 캡처: 3NF 최종 4개 테이블

	cust_id	cust_name
1	C01	김하늘
2	C02	이준호

PRODUCTS — PK: prod_id

prod_id (PK)	prod_name	unit_price
P10	노트북	1200000
P22	마우스	25000

SELECT * FROM products ORDER BY prod_id; 348 ms

Output: telecom.public.products

	prod_id	prod_name	unit_price
1	P10	노트북	1200000
2	P22	마우스	25000

ORDERS — PK: order_id · FK: cust_id → CUSTOMERS.cust_id

order_id (PK)	order_dt	cust_id (FK)
O1001	2024-06-01	C01
O1002	2024-06-03	C02

SELECT * FROM orders ORDER BY order_id; 372 ms

Output: telecom.public.orders

	order_id	order_dt	cust_id
1	O1001	2024-06-01	C01
2	O1002	2024-06-03	C02

ORDER_ITEMS — PK: (order_id, prod_id) · FK: order_id → ORDERS.order_id, prod_id → PRODUCTS.prod_id

order_id (PK,FK)	prod_id (PK,FK)	qty
O1001	P10	1
O1001	P22	2
O1002	P10	1

```
SELECT * FROM orders ORDER BY order_id; 372 ms
```

Output **telecom.public.orders**

	order_id	order_dt	cust_id
1	01001	2024-06-01	C01
2	01002	2024-06-03	C02

이상현상(Anomaly) 해소 근거

갱신 이상: 분해 전에는 '노트북' 단가 변경 시 해당 상품이 포함된 모든 주문행을 수정해야 했지만, 분해 후에는 PRODUCTS 1행만 수정하면 된다.

삽입 이상: 분해 전에는 주문이 없으면 신규 상품·고객을 등록할 수 없었지만, 분해 후에는 PRODUCTS/CUSTOMERS에 주문 없이도 독립적으로 등록 가능하다.

삭제 이상: 분해 전에는 마지막 주문행을 삭제하면 상품·고객 정보까지 함께 사라졌지만, 분해 후에는 ORDER_ITEMS/ORDERS 삭제와 무관하게 PRODUCTS/CUSTOMERS 정보가 보존된다.

5. 단일 조회 쿼리 — 과제 2

대상 테이블: customer (고객 마스터, public 스키마, LAB3 적재분)

요구사항 반영

1. WHERE ... IN (...) AND ... — grade가 GOLD 또는 VIP이고, 2020년 이후 가입한 고객만
2. AGE(birth_ymd)로 만 나이 계산
3. CASE로 연령대 라벨(30 미만 '청년' / 30~49 '중년' / 50 이상 '장년')
4. COALESCE(region, '미상')로 NULL 지역 대체
5. ORDER BY join_dt ASC LIMIT 15 — 가입일 오래된 순 상위 15건

```
SELECT
  cust_id,
  name,
  grade,
  EXTRACT(YEAR FROM AGE(birth_ymd)) AS age,
  CASE
    WHEN EXTRACT(YEAR FROM AGE(birth_ymd)) < 30 THEN '청년'
    WHEN EXTRACT(YEAR FROM AGE(birth_ymd)) BETWEEN 30 AND 49 THEN '중년'
    ELSE '장년'
  END AS age_group,
  COALESCE(region, '미상') AS region,
  join_dt
FROM customer
WHERE grade IN ('GOLD', 'VIP')
  AND join_dt >= DATE '2020-01-01'
```



```
ORDER BY join_dt ASC
LIMIT 15;
```

SQL Query Editor:

```
SELECT
  cust_id,
  name,
  grade,
  EXTRACT(YEAR FROM AGE(birth_ymd)) AS age,
  CASE
    WHEN EXTRACT(YEAR FROM AGE(birth_ymd)) < 30 THEN '청년'
    WHEN EXTRACT(YEAR FROM AGE(birth_ymd)) BETWEEN 30 AND 49 THEN '중년'
    ELSE '장년'
  END
```

customer 1 X

Enter a SQL expression to filter results (use Ctrl+Space)

	123 cust_id	AZ name	AZ grade	123 age	AZ age_group	AZ region	join_dt
1	370	고객370	GOLD	8	청년	인천	2022-01-05
2	180	고객180	VIP	31	중년	서울	2022-01-06
3	314	고객314	VIP	42	중년	인천	2022-01-08
4	428	고객428	VIP	54	장년	대전	2022-01-10
5	345	고객345	GOLD	20	청년	강원	2022-01-16
6	446	고객446	GOLD	34	중년	서울	2022-01-22
7	378	고객378	GOLD	9	청년	대구	2022-02-01
8	90	고객90	GOLD	17	청년	서울	2022-02-17
9	259	고객259	GOLD	52	장년	서울	2022-02-27
10	76	고객76	GOLD	54	장년	서울	2022-03-01
11	200	고객200	GOLD	22	청년	경기	2022-03-08
12	28	고객28	GOLD	57	장년	부산	2022-03-09
13	162	고객162	VIP	38	중년	강원	2022-03-12
14	362	고객362	GOLD	59	장년	부산	2022-03-23
15	54	고객54	GOLD	43	중년	경기	2022-03-30

6. PostgreSQL 접속 결과 화면

```
[→ db-sql-tuning git:(main) × psql postgres
psql (17.10 (Homebrew))
도움말을 보려면 "help"를 입력하십시오.

postgres=# SELECT version();
postgres=#
```

```
version

-----
PostgreSQL 17.10 (Homebrew) on aarch64-apple-darwin25.4.0, compiled by Apple clang version 21.0.0 (clang-2100.0.123.102), 64-bit
(1개 행)

(END)
```

The screenshot shows a SQL IDE interface. At the top, a query editor contains the SQL statement: `SELECT count(*) FROM customer;`. Below the editor, a tab labeled "customer 1" is active. The results pane shows the execution of the query. The top of the results pane displays the query text: `SELECT count(*) FROM custo`. Below this, a table is shown with the following structure:

	123 count
1	500

The table has two columns: an index column and a column named "123 count". The first row of the table shows the value "500" for the "123 count" column. The results pane also includes a vertical toolbar on the left with icons for "Grid", "Text", and "Record".

7. 문항별 SQL문 + 실행 결과 화면

과제 2 실행 결과

```
SELECT
cust_id,
name,
grade,
EXTRACT(YEAR FROM AGE(birth_ymd)) AS age,
CASE
WHEN EXTRACT(YEAR FROM AGE(birth_ymd)) < 30 THEN '청년'
WHEN EXTRACT(YEAR FROM AGE(birth_ymd)) BETWEEN 30 AND 49 THEN '중년'
ELSE '장년'
END AS age_group,
region AS region,
join_dt
FROM customer1
```

	123 cust_id	AZ name	AZ grade	123 age	AZ age_group	AZ region	② join_dt
1	370	고객370	GOLD	8	청년	인천	2022-01-05
2	180	고객180	VIP	31	중년	서울	2022-01-06
3	314	고객314	VIP	42	중년	인천	2022-01-08
4	428	고객428	VIP	54	장년	대전	2022-01-10
5	345	고객345	GOLD	20	청년	강원	2022-01-16
6	446	고객446	GOLD	34	중년	서울	2022-01-22
7	378	고객378	GOLD	9	청년	대구	2022-02-01
8	90	고객90	GOLD	17	청년	서울	2022-02-17
9	259	고객259	GOLD	52	장년	서울	2022-02-27
10	76	고객76	GOLD	54	장년	서울	2022-03-01
11	200	고객200	GOLD	22	청년	경기	2022-03-08
12	28	고객28	GOLD	57	장년	부산	2022-03-09
13	162	고객162	VIP	38	중년	강원	2022-03-12
14	362	고객362	GOLD	59	장년	부산	2022-03-23
15	54	고객54	GOLD	43	중년	경기	2022-03-30